What is a Semaphore?

- **Semaphore** was proposed by **Dijkstra** to manage **multiple processes** working together (concurrent processes).
- It helps **solve mutual exclusion problems** (making sure only one process enters a critical section at a time).
- **POSIX Semaphores** are used for **synchronization between threads** or **between processes** (like a mutex).
- A **semaphore is an integer variable**:
    - It can be **incremented** (to unlock).
    - It can be **decremented** (to lock).
    - Threads or processes **increase or decrease** its value to control access.

POSIX Types of Semaphores

Type	Description
Named Semaphore	Semaphore with a name, visible across different processes.
Memory-mapped (Unnamed) Semaphore	Semaphore created in shared memory, used between related processes or threads.

Unnamed Semaphores (Memory-based Semaphores)

- No name is associated with these semaphores.
- Provides synchronization between threads or related processes.
- Stored in shared memory.
- For threads, simply declare it as a **global variable**.

Kinds of Semaphores

Kind	Description
Binary Semaphore	Values 0 or 1 only. Acts like a simple lock.
Counting Semaphore	Holds values 0,1,2,3,... and allows multiple threads to access a resource.

Uses of Semaphores

Semaphores are used for:

1. **Placing Locks**

- **Counting Semaphore**: Allow **limited number** of threads.
- **Binary Semaphore**: Allow **only one** thread.

2. **Condition Variables**
    - Threads can **wait for a condition** and **signal** each other.
    - Useful when thread needs to wait until some work is done by another thread.

Semaphores also **prevent deadlocks** by controlling thread access order.

Real-World Scenarios where Semaphore is Needed

Scenario	Why Semaphore is Needed?
Printer Access	Multiple users want to use the same printer. Only one print job should be handled at a time. Semaphore controls access.
Parking Lot System	Limited number of parking spaces. Semaphore ensures only the allowed number of cars can park at a time.

Where Mutex is NOT Enough — Semaphore Needed

Case	Why Mutex fails	Why Semaphore works
Multiple printers or resources (more than 1)	Mutex only allows 1 thread at a time — cannot allow multiple threads together.	**Counting Semaphore** can allow 2, 3, or more threads depending on available printers/resources.
Waiting for task completion (Condition Sync)	Mutex only locks/unlocks — no way to "wait until condition is true."	**Binary Semaphore** can make a thread wait and another thread signal it to continue.

In short:

- Mutex = Simple lock/unlock
- Semaphore = Lock, unlock **+** signal/wait capabilities!

Semaphore vs Mutex (Differences)

Feature	Semaphore	Mutex
Values	Semaphore can have 0,1,2,3...	Mutex is 0 (unlocked) or 1 (locked).

Who Unlocks	Any thread can unlock a semaphore.	Only the locking thread can unlock the mutex.
Purpose	Locking + Waiting (Condition Handling)	Only Locking
Flexibility	More flexible (condition signaling).	Only mutual exclusion.
Persistence	Named Semaphores are **kernel persistent**; unnamed are **process persistent**.	Process persistent.

Semaphore System Calls (Functions)

Function Name	Purpose
`sem_init()`	Initialize the semaphore.
`sem_wait()`	Lock the semaphore (decrement). Block if value is 0.
`sem_trywait()`	Try to lock. If not possible, return error.

`sem_post()`	Unlock the semaphore (increment).
`sem_destroy()`	Destroy and clean up semaphore.

Create a Semaphore

Syntax:

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Parameter	Meaning
`sem`	Semaphore variable.
`pshared`	0 → Between threads. Non-0 → Between processes.
`value`	Initial value (e.g., 0 or 1).

Example:

```
sem_t s;
```

```
sem_init(&s, 0, 1); // Shared between threads,
initially free
```

Semaphore Operations

Function	Description
`sem_wait(sem_t *sem)`	Decrease value. If 0, wait (block).
`sem_post(sem_t *sem)`	Increase value. Wake up a waiting thread if any.
`sem_trywait(sem_t *sem)`	Try to decrease without blocking.
`sem_destroy(sem_t *sem)`	Destroy semaphore and free resources.

Examples

Example 1: Binary Semaphore as Lock

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem;

void* thread_function(void* arg) {
    sem_wait(&sem);
    printf("Thread %ld got the lock!\n", pthread_self());
    sleep(2);
    sem_post(&sem);
    pthread_exit(NULL);
}

int main() {
    pthread_t tid1, tid2;
    sem_init(&sem, 0, 1);

    pthread_create(&tid1, NULL, thread_function, NULL);
    pthread_create(&tid2, NULL, thread_function, NULL);
```

```
    pthread_exit(NULL);

}
```

Expected Output

- `Thread 1234567 got the lock!`
- `Thread 7654321 got the lock!`

Only one thread enters at a time.

Example 2: Binary Semaphore as Condition Variable

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>


sem_t sem;


void* child_function(void* arg) {
    printf("Child: begin\n");
    sem_post(&sem); // Child signals
    printf("Child: end\n");
    pthread_exit(NULL);
}


int main() {
    pthread_t tid;
    sem_init(&sem, 0, 0); // Start locked


    printf("Parent: begin\n");


```

```
    pthread_create(&tid, NULL, child_function, NULL);


    sem_wait(&sem); // Parent waits


    printf("Parent: end\n");


    pthread_exit(NULL);
}
```

Expected Output

- `Parent: begin`
- `Child: begin`
- `Child: end`
- `Parent: end`

Parent waits until Child finishes.

FINAL SUMMARY

- **Semaphore** = Synchronization tool using counter.
- **Types**: Named and Unnamed.
- **Kinds**: Binary (lock/unlock) and Counting (multiple access).
- **Used for**:
    - Locking critical sections.
    - Waiting for conditions.

- **Where Mutex fails**:
    - Multiple accesses (counting semaphore needed)
    - Waiting for another thread's work (binary semaphore needed)

Semaphore = Lock + Signal + Count
Mutex = Only Lock/Unlock
